

SourceBridge

One source. Multiple formats. Traceable facts.

Team	BYTE ME · Team ID 155404
Team lead	Mohammad Saad
Members	Jatin Baghel · Ricky Mukherjee · Dhruv Vishwakarma · Hanzala Rafat · Yogita Verma
Event	Smart India Hackathon 2026 · Problem statement 26154
Theme	Blockchain & Cybersecurity
Repository	github.com/saad4228/sourcebridge

1. What it does

An organisation receives one document — an incident report, a study, a policy note — and has to communicate it to several different audiences at once. A one-page summary for leadership. A public advisory. A briefing deck. A social post. A video script.

Written separately, those versions drift apart. A figure gets rounded. A caveat gets dropped. "*Attribution remains unconfirmed*" quietly becomes "*attribution confirmed*". The number is rarely wrong — what changes is the **certainty around it**, and that is the part nobody notices.

SourceBridge reads the source **once**, extracts its facts, figures and caveats into a shared ledger, and writes every output from that same foundation. The figures cannot diverge, because they all come from one place. And every claim carries a link back to the exact passage that supports it.

2. What it produces

Seven formats, from one upload:

Executive summary	Finding, evidence, implications, decisions
LinkedIn post	Hook, explanation, takeaway, optional call to action
X post or thread	A single post, or a thread that splits itself correctly
Advisory	Formal notice with actions and caveats
Presentation	Slides with speaker notes — downloads as <code>.pptx</code>
Infographic	Headline and key messages — downloads as <code>.svg</code>
Video package	Script, storyboard, narration, subtitles — and a rendered <code>.mp4</code>

Each is a real generator with its own output schema and a preview shaped like the thing it becomes. None is a placeholder.

3. Why this is not a chatbot wrapper

Three properties, each enforced by the architecture rather than by asking a model nicely.

One analysis, seven consumers. Extraction and fact analysis run once; every format reads the same ledger. Asking a chatbot seven times produces seven independent answers, and independence is exactly what causes drift. The design removes it.

The model never draws a figure. It returns structured data. Slides, infographics and every video frame are drawn by the application's own code — so a model cannot render 486 Gbps incorrectly, because it is never asked to render anything. Downloads involve no AI call at all.

Qualifiers are checked, not assumed. Comparing numbers catches a figure that changed. It cannot catch a figure that stayed while the certainty around it did not. SourceBridge flags a dropped "*preliminary*" as a warning, and an escalated "*some*" → "*all*" or "*unconfirmed*" → "*confirmed*" as an error.

Every export also ships a **provenance record** — a SHA-256 hash chain over the source, the fact ledger and each artefact, so altering any of them afterwards is detectable. This is a hash chain, not a blockchain: it proves integrity and ordering, not that the original document was authentic.

4. Running it yourself

What you need

- **Node.js 20 or newer** — nodejs.org. Check with `node --version`.
- **At least one free API key** (below). No billing, no credit card.
- **ffmpeg** — optional, only for rendered video.

Step 1 — Get an API key

Both free tiers are sufficient. Getting both takes about two minutes and gives the best result.

Provider	Where	What it powers
Groq	console.groq.com/keys	Text generation. Tried first — answers in about a second.
Google Gemini	aistudio.google.com/apikey	Reading images and video, and speaking video narration.

- **Both keys** — fastest, every feature available.
- **Gemini only** — everything works, more slowly.
- **Groq only** — all seven formats generate, but image and video *sources* are refused.

Step 2 — Download and install

```
git clone https://github.com/saad4228/sourcebridge.git
cd sourcebridge
npm install
```

If you do not have Git, download the repository as a ZIP from GitHub and extract it instead.

Step 3 — Add your keys

Copy the example file:

```
cp .env.example .env.local
```

Open `.env.local` in any text editor and fill in the keys you have:

```
GROQ_API_KEY=your-key-here
GEMINI_API_KEY=your-key-here
```

`.env.local` is never committed to the repository. Keys are read **only on the server** and are never sent to the browser.

Step 4 — Start it

```
npm run dev
```

Open **http://localhost:3000** in a browser.

Step 5 — Try it

Press **Try the sample incident report**. A synthetic cyber-incident assessment is bundled, so you do not need a document of your own to see the whole pipeline work.

Optional — rendered video

Only the `.mp4` export needs `ffmpeg`. Everything else works without it, and the application tells you so rather than failing silently.

```
winget install Gyan.FFmpeg      Windows
brew install ffmpeg           macOS
sudo apt install ffmpeg       Debian / Ubuntu
```

If something goes wrong

Symptom	Cause and fix
"No API key found"	<code>.env.local</code> is missing or empty. Restart after editing it — keys are read at startup.
Generation fails with a quota error	A free-tier limit. Groq's clears in about a minute; Gemini's daily allowance resets after 24 hours. Other models are tried automatically.
"Rendering an MP4 needs ffmpeg"	<code>ffmpeg</code> is not installed, or set <code>FFMPEG_PATH</code> to the binary. The <code>.zip</code> package still works.
Scanned PDF refused	Deliberate. There is no OCR — paste the text, or upload a photo of the page, which <i>is</i> supported.
Port 3000 already in use	Stop the other process, or run <code>npm run dev -- -p 3001</code> .

5. Using it

1. **Provide a source** — a text-based PDF, an image, a short video, a web link, or pasted text. 2. **Review the extraction**. Each passage is shown with a stable ID, page by page. 3. **The shared fact ledger builds automatically** — claims, figures with units, dates and caveats, each linked to the passages supporting it. 4. **Set the brief** — audience, objective, tone, language, level of detail. These are instructions, not labels: the same source under different settings genuinely reads differently. 5. **Choose formats and generate**. Each runs independently, so one failure does not lose the rest. 6. **Inspect the evidence**. Click any evidence chip to read the source passage behind a claim. 7. **Review the findings**. Structural checks report unverified figures, dropped qualifiers and escalated claims for a human to judge. 8. **Edit anything**, then download. Exports always use your edited version. Outputs download individually, or all at once as an archive including the provenance record.

6. What it deliberately does not do

Stated plainly, because the interface is not permitted to imply otherwise.

- **It does not verify facts.** The checks are structural: evidence IDs resolve, figures appear in the source, qualifiers survive, content fits its layout. Whether a claim is *true* is not assessed.
- **It does not read scanned PDFs.** No OCR. Scanned files are refused with a clear message. Images are supported, read by a vision model — a transcription, not a literal extraction, and labelled so.
- **It does not generate imagery or video with AI.** Every frame is drawn by the application's code. A generative model cannot be trusted with a figure, and in a video the viewer cannot check it.
- **It does not show confidence scores.** Any number shown would be invented.
- **It does not store anything.** No accounts, no database. Work lives in the browser tab and is lost on refresh; the app warns before you leave.
- **Multilingual output is generated but not quality-checked.** Non-English works and the exporters declare fonts covering Devanagari, Bengali, Tamil and Telugu, but no claim is made about translation quality.

7. Further reading

- `docs/ARCHITECTURE.md` — the two-page technical architecture: pipeline, guarantees, stack, security model, and how the model chain was chosen by measurement.
- `README.md` — developer-facing setup, commands and test instructions.
- **247 automated tests** run without an API key: `npm test`.

Sample documents in `samples/` are synthetic. The incident, the organisations and every figure they describe are fictional.